Algoritmos de busca de caminho em representações navegáveis tridimensionais

Uma análise comparativa e estatística

André Vitor Bastos de Macêdo
Orientador: Prof. Paulo César Rodacki Gomes

Instituto Federal Catarinense
Campus Blumenau

2026

INSTITUTO FEDERAL
Catarinense
Campus Blumenau

Sumário

01 Introdução e Objetivos

02 Fundamentação Teórica

03 Metodologia e Desenvolvimento

04 Resultados e Discussões

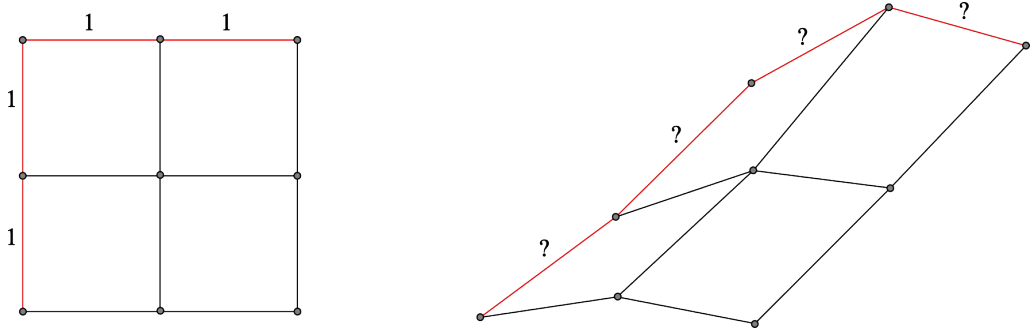
05 Conclusões e Trabalhos futuros

Busca de caminhos em espaços 3D

Busca de caminhos (*pathfinding*) é encontrar o caminho de menor custo entre dois pontos explorando um grafo [2]. É um componente central de jogos e simulações em tempo real [11].

Em uma grade 2D uniforme, o custo entre vizinhos é constante.
Em um terreno 3D, o custo depende da variação de altura, e a própria forma de transformar o terreno em grafo passa a ser uma decisão.

Figura 1: Comparativo de representações 2D e 3D



Pergunta de pesquisa e contribuições

"Como a **representação navegável** de um terreno 3D e as **características do terreno** influenciam o tempo de execução e a qualidade dos caminhos encontrados por algoritmos clássicos de busca?"

Contribuições:

1. **Comparação entre representações navegáveis.**
   As mesmas instâncias de terreno são convertidas em quatro grafos diferentes e avaliadas com os mesmos algoritmos:
   - a partir do mapa de altura: grade de altura (2.5D) e superfície voxel;
   - a partir da malha poligonal: grafo de vértices e grafo dual.
2. **Metodologia experimental reprodutível.**
   Os terrenos são gerados proceduralmente a partir de parâmetros controlados (escala, lacunaridade, persistência) e semente registrada, e os resultados passam por análise estatística (ANOVA e testes post hoc).
3. **Implementação open-source.**
   Inclui uma biblioteca de grafos leve e o motor gráfico IFCG, com visualização das malhas e dos caminhos.

O trabalho não propõe um novo algoritmo de busca: seu foco é a avaliação comparativa.

Objetivos

Objetivo geral:

Analisar comparativa e estatisticamente o desempenho de algoritmos clássicos de busca de caminhos em diferentes representações navegáveis de terrenos 3D.

Objetivos específicos:

1. Implementar uma biblioteca de grafos e os algoritmos Dijkstra, A*, Greedy Best-First e Weighted A*.
2. Desenvolver um gerador procedural de terrenos baseado em ruído de Perlin, parametrizado por escala, lacunaridade e persistência.
3. Construir quatro representações navegáveis a partir do terreno: grade de altura, superfície voxel, grafo de vértices e grafo dual da malha.
4. Desenvolver um ambiente 3D com execução concorrente para visualização das malhas e dos caminhos encontrados.
5. Automatizar a coleta de métricas e analisar estatisticamente os resultados obtidos.

Por que um motor próprio?

Trabalhos relacionados avaliam algoritmos em uma única representação:
voxels em Johansson (2024) [5], grades 2D em Kapi (2022) [6].

Comparar representações exige controlar como a malha e o grafo são construídos.

Motores consolidados (Unity, Unreal)	Motor próprio (IFCG)
Os sistemas de navegação embutidos geram sua própria malha de navegação, e a representação não é uma variável controlável.	As quatro representações são construídas explicitamente a partir do mesmo terreno.
Rotinas internas da engine competem pela CPU durante a execução.	A coleta de dados roda isolada da renderização, só com o código medido.
Os algoritmos de busca vêm prontos, com implementações e otimizações distintas.	Todos os algoritmos operam sobre o mesmo grafo: as diferenças vêm da representação e do algoritmo, não da implementação.

O motor serve à visualização e à validação dos caminhos, as medições são feitas fora do laço de renderização.

Algoritmos de busca de caminhos

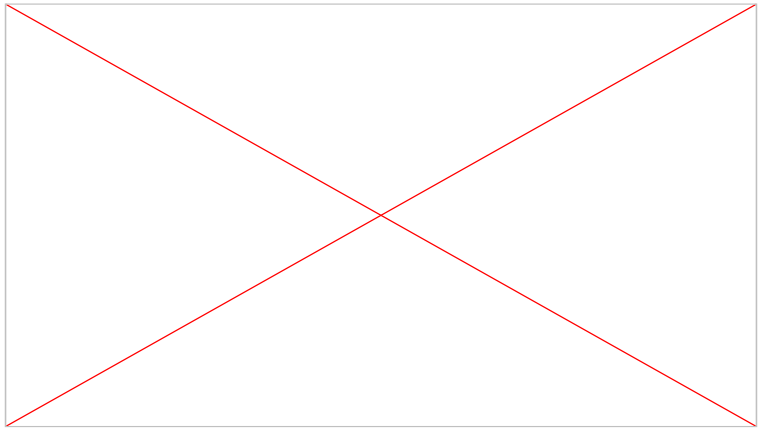
Algoritmo	Função de avaliação	Garantia
Dijkstra	$f(n) = g(n)$	Caminho ótimo
A*	$f(n) = g(n) + h(n)$	Caminho ótimo
Weighted A* (WA*)	$f(n) = g(n) + w \cdot h(n)$	Custo ≤ w: ótimo
Greedy Best-First (GBFS)	$f(n) = h(n)$	Nenhuma

g(n): custo acumulado do início até n
h(n): estimativa do custo de n até o objetivo (heurística)
w: peso constante da heurística

Heurística usada: distância euclidiana 3D → $\sqrt{(\Delta x^2 + \Delta y^2 + \Delta z^2)}$

Geração procedural com ruído de Perlin

O terreno é um mapa de altura gerado pela soma de camadas (oitavas) de ruído de Perlin [13].
A cada oitava, a frequência é multiplicada pela **lacunaridade** e o peso pela **persistência** [12].



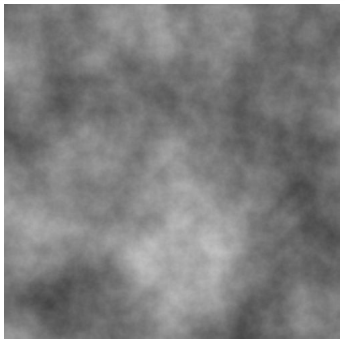
Lacunaridade l = 2, persistência p = 0,5. Contraste ajustado para visualização.

Geração procedural com ruído de Perlin

Lacunaridade:
o quão fino é o detalhe adicionado a cada oitava.

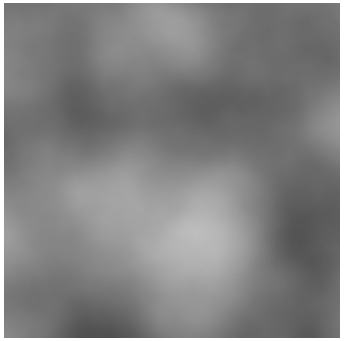


l = 1,5

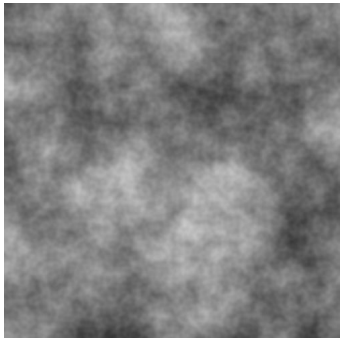


l = 2,5

Persistência:
o quanto esse detalhe pesa no relevo final.



p = 0,3



p = 0,7

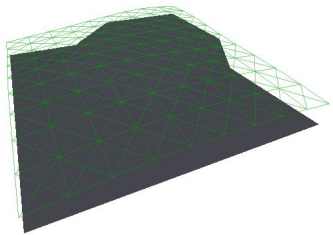
Mesma semente em todos os mapas. Contraste ajustado para visualização.

Representações navegáveis

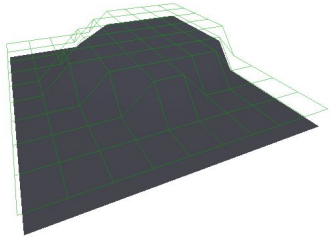
O mesmo terreno é convertido em quatro grafos de navegação, em dois grupos.

A partir do mapa de altura

Grade de 2.5D vizinhança-8

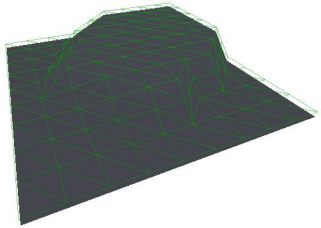


Superfície voxel vizinhança-4

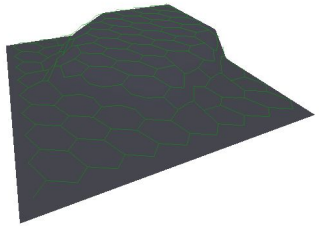


A partir da malha poligonal

Grafo de vértices da malha

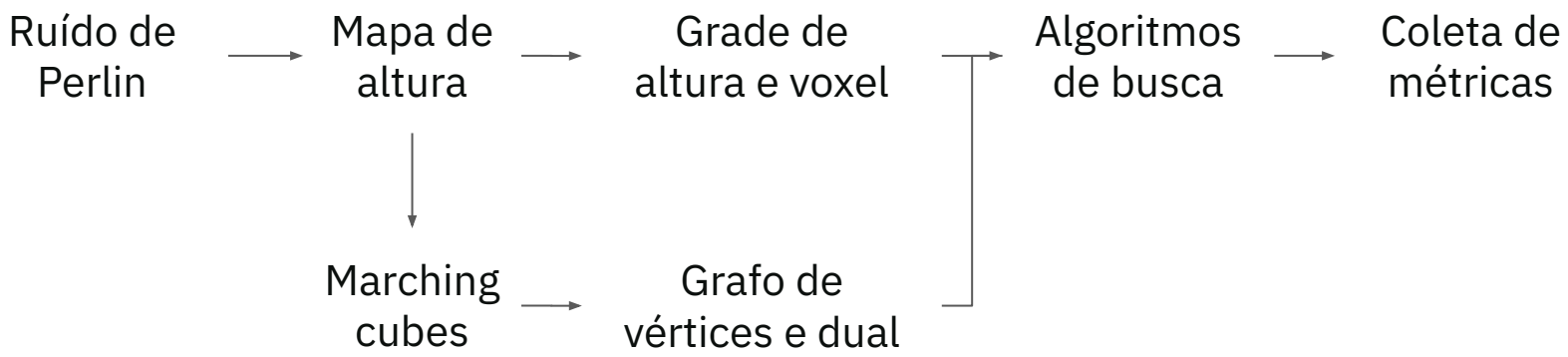


Grafo dual de polígonos



Em todas: peso da aresta = distância euclidiana 3D; arestas com inclinação acima de 45° são descartadas.

Pipeline experimental



Desenho experimental

Independentes	Dependentes	Controladas
Resolução* N × N: 500 a 800, em passos de 100	**Grafo:** tempo de construção, nós, arestas, memória	**Oitavas:** 6
Lacunaridade*: 1,5 a 2,5	**Busca:** tempo, nós expandidos, memória estimada	**Proporção do relevo**: altura/onda = 2
Persistência*: 0,3 a 0,7	**Caminho:** custo, comprimento geométrico	**Inclinação máxima:** 45°
Representações navegáveis: Grade de altura, superície voxel, grafo de vértices e grafo dual		**Pares início/fim:** 10 por terreno
Algoritmos: Dijkstra, A*, WA*, GBFS		**Heurística:** Euclidiana 3D

* Cada fator de terreno é variado isoladamente, com os demais fixos. Repetições: 10 terrenos por nível

Análise estatística

Variáveis de resposta:

- **Tempo de busca**
- **Nós expandidos:** esforço da busca, independente do hardware
- **Custo relativo:** custo do caminho ÷ custo ótimo (Dijkstra)

Fatores:

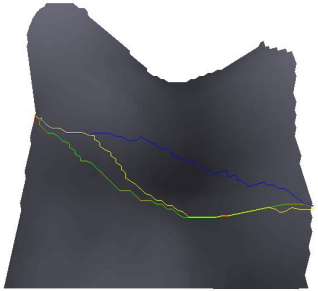
- Algoritmos
- Representação
- Nível do fator de terreno.

Procedimento

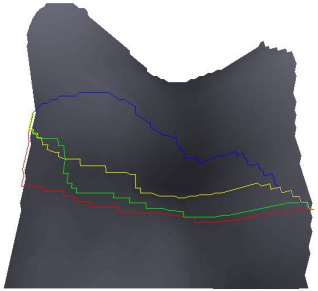
Verificação de normalidade e homogeneidade → ANOVA ($\alpha = 0,05$) → Post hoc Tuckey

Caminhos encontrados

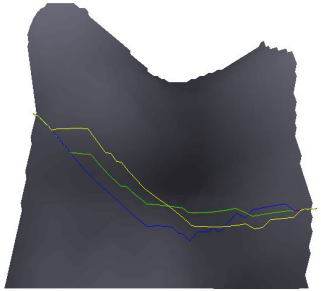
Grade de 2.5D vizinhança-8



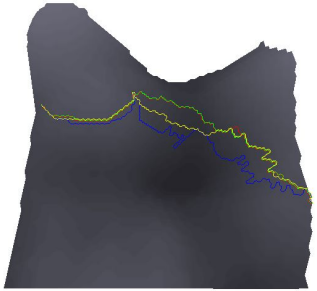
Superfície voxel vizinhança-4



Grafo de vértices da malha



Grafo dual de polígonos



● Dijkstra; ● A*; ● WA*; ● GBFS.

Mesmo terreno, mesmo par início/fim e mesma câmera nas quatro representações.